Text as Data

Class 5: Supervised text classification

Petro Tolochko

CEU

Fall 2026

Where we are in the course

- Counts and multinomial models gave us a representation of language.
- The Federalist Papers showed how class-specific word distributions can distinguish authors.
- Dictionaries gave us explicit rules and a way to check their mistakes.
- Today: learn a classification rule from labelled texts.

Supervised and unsupervised learning

Supervised

An outcome is defined and examples have labels.

Learn a function that predicts that outcome for new texts.

Unsupervised

No outcome label is supplied.

Look for structure, such as groups of similar documents or recurring topics.

Both require substantive interpretation. Today we know which label we want to predict.

Classification and regression

Task	Outcome	Text example
Classification	A pre-existing category	Positive/negative review; policy frame; spam/not spam
Regression	A numerical quantity	Review rating; expert-assigned ideology score

Both learn from pairs of features and outcomes. The output and fitting objective differ.

Supervised text scaling requires reference scores or anchors. A position inferred without labelled scores is a different modelling task.

Hand-written weights and learned weights

A sentiment dictionary assigns positive and negative weights by hand:

$$s_D(d) = b + \sum_w a_w x_w.$$

A linear classifier has a similar form:

$$z(d) = b + \sum_w \beta_w x_w.$$

The difference is where the weights come from. A supervised model estimates them using examples with known labels.

Dictionaries and supervised models

Dictionary rule	Supervised model
Terms and weights chosen by a researcher	Weights or other parameters estimated from labels
Can run without fitting to labelled examples	Requires labelled training examples
The matching rule is directly inspectable	Interpretability depends on the model and features
Adapt vocabulary and rules manually	Refit on examples from a new setting

Both still need validation. A model adapts to the labels and patterns available in its training sample.

Prediction and measurement

Supervised learning asks whether a rule predicts labels for new texts.

Our substantive question is whether those labels measure the concept we want to study.

A model can reproduce a source's labels while inheriting their errors or omissions. A higher prediction score does not settle the measurement question.

Grimmer, Roberts, and Stewart (2022), *Text as Data*.

Our running example: Rotten Tomatoes

- 10,662 movie-review snippets from Pang and Lee's sentence-polarity dataset
- 5,331 positive and 5,331 negative source-review labels
- Same corpus and small sentiment dictionary as Class 4
- Our comparison: can learned word weights improve on that dictionary?

The label comes from the source review's *fresh* or *rotten* status. It is not an independent human judgement of every isolated snippet.

Pang and Lee (2005), sentence-polarity dataset v1.0.

A supervised text workflow

1. Define the outcome and obtain labelled examples.
2. Separate development data from the final test data.
3. Fit a text representation and learn a rule linking features to labels.
4. Choose settings and assess the final rule on unseen examples.
5. Apply the fitted rule to new texts within its validated scope.

Training estimates the mapping. Validation and substantive interpretation determine which claims that mapping can support.

The model sees a feature vector

Illustrative text	good	not	dull
good	1	0	0
not good	1	1	0
dull and not good	1	1	1

A count vector records which terms occur and how often. Most entries in a large vocabulary are zero.

A unigram representation omits order. Adding adjacent word pairs gives a feature for *not good*, but does not automatically teach the model what negation means.

Counts and TF-IDF

Counts

Each entry is the number of times a word occurs.

This matches the multinomial word model used by Naive Bayes.

TF-IDF

Weight a word's frequency by how widely it occurs in the training corpus.

The notebook uses TF-IDF for logistic regression and normalizes each document vector to length one.

Both are choices about what the model sees. The vocabulary and corpus frequency weights must be learned from training data.

Representation, selection, and classifier

- **Representation:** counts, TF-IDF, word pairs, or another numerical description of a document.
- **Feature selection:** decide which terms or patterns can enter the model.
- **Classifier:** decide how the available features will be combined to predict a label.

Dropping rare terms can reduce noise. Dropping function words can remove useful signals such as negation.

Choose these settings using development data. Feature selection that uses labels must also remain inside the training folds.

What supervised learning estimates

The labelled training set is

$$\mathcal{D}_{\text{train}} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}.$$

A model produces a prediction or score $f(\mathbf{x}; \theta)$. Training estimates its parameters:

$$\hat{\theta} = \arg\min_{\theta} \mathcal{L}(\theta; \mathcal{D}_{\text{train}}).$$

The objective function says what counts as a good fit. Evaluation asks how the fitted model performs on texts it did not learn from.

Squared-error loss for regression

For a numerical outcome, a common objective is mean squared error:

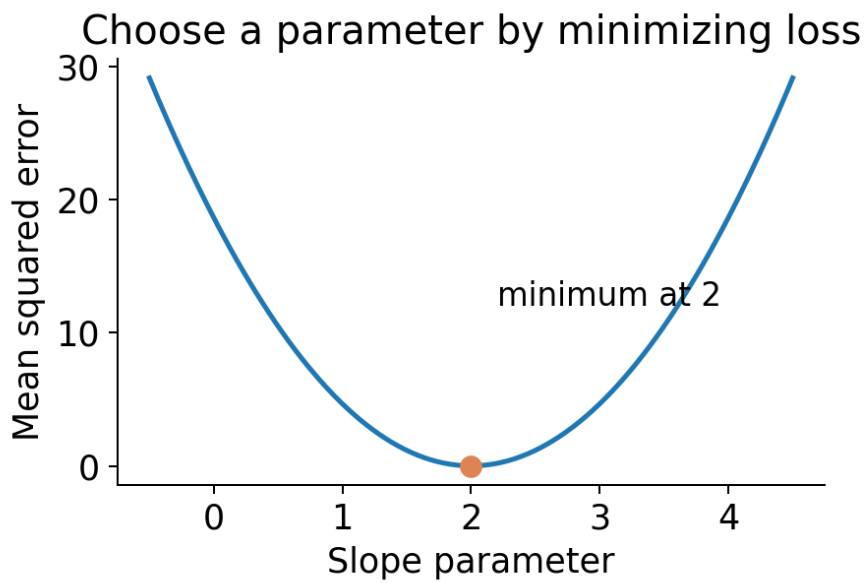
$$\mathrm{MSE}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\left[y_i - f(\mathbf{x}_i;\theta)\right]^2.$$

Illustrative data: $x = (1, 2, 3)$ and $y = (2, 4, 6)$.

For $f_\theta(x) = \theta x$:

$$\mathrm{MSE}(\theta) = \frac{14}{3}(\theta - 2)^2.$$

The minimum is at $\theta = 2$.



Optimization means reducing the loss

One approach is gradient descent: move parameters in the direction that lowers the objective.

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_\theta \mathcal{L}(\theta^{(t)}).$$

For the regression example, the derivative is

$$\frac{\partial \text{MSE}}{\partial \theta} = \frac{28}{3}(\theta - 2).$$

Starting at $\theta = 0$ with step size $\eta = 0.1$ gives a first update to $\theta \approx 1.867$.

The model defines predictions, the loss defines a good fit, and the optimizer searches for parameters. Our logistic implementation uses a more elaborate optimizer.

Training sample coverage

Labels can come from human coding, existing ratings, or metadata linked to the text.

Whatever their source, the training sample must contain the distinctions we want the classifier to learn.

- Cover relevant classes, sources, periods, and writing styles.
- Include difficult cases and examples near category boundaries.
- Check whether a label describes the text itself or a larger object, as with our review-level sentiment labels.

How many labelled documents?

There is no universal sample size or percentage that makes a text classifier reliable.

- More categories and rarer categories need broader label coverage.
- Noisy labels and a diverse target corpus make learning harder.
- Many possible features increase the need for regularization and adequate examples.

A learning curve fits the same procedure on increasing training sizes and measures performance on separate validation data.

It helps assess whether more labels improve prediction or whether the representation and label definition need attention.

Training, validation, and test data

Split	Snippets	Purpose
Training	6,396	Estimate word probabilities or coefficients
Validation	2,133	Choose smoothing, regularization, and representation
Test	2,133	Final comparison after choices have been fixed

The notebook uses a seeded, stratified split, approximately 60% / 20% / 20%.

Once settings are chosen, we refit on the 8,529 development snippets and evaluate on the shared test set.

The split follows the research question

Target claim	Suitable evaluation design
New texts from the same population	Random held-out documents, if observations are independent
New authors, sources, or reviews	Keep each group within one split
Future political language	Train on earlier texts and test on later texts

Stratification preserves class balance. It does not stop duplicates or related texts from leaking across splits.

Our Rotten Tomatoes files lack review and author identifiers, so this draft demonstrates a snippet-level split.

Leakage through preprocessing

Even without using test labels, we can leak information from held-out text into development.

- Computing vocabulary and IDF values on the full corpus uses the test set.
- Choosing terms after reading test errors lets the test set guide the rule.
- Repeatedly trying settings against test scores turns the test set into development data.

The notebook uses pipelines: each fit learns the representation and classifier together from the appropriate training subset.

Cross-validation

Run	Fold 1	Fold 2	Fold 3
1	Validate	Train	Train
2	Train	Validate	Train
3	Train	Train	Validate

$$\widehat{M}_{\mathrm{CV}} = \frac{1}{3}\sum_{j=1}^{3} M_j.$$

Fit the vocabulary, IDF, and classifier afresh inside each run.

Cross-validation can replace a single validation split for model selection. A separate final test set is still needed.

Baselines give a comparison meaning

Method	What it contributes
Majority class	Performance without looking at text
Dictionary	A transparent rule based on selected words
Naive Bayes	Class-specific word distributions learned from counts
Logistic regression	Weights learned directly to predict labels
Linear SVM	A separating boundary learned with a margin-based loss

Use the same test examples and label definition for every method. A different split can change the apparent winner.

The multinomial model revisited

Each class c has a distribution over a fixed vocabulary:

$$\boldsymbol{\theta}_c = (\theta_{c1}, \ldots, \theta_{cV}), \qquad \sum_{w=1}^{V} \theta_{cw} = 1.$$

For document counts $\mathbf{x}$ with $n = \sum_w x_w$:

$$P(\mathbf{x} \mid c, n) = \frac{n!}{\prod_w x_w!} \prod_w \theta_{cw}^{x_w}.$$

In the Federalist exercise, classes were authors. Here, classes are positive and negative source-review labels.

Bayes' rule

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}.$$

- $P(A)$: prior probability of the hypothesis
- $P(B \mid A)$: likelihood of the evidence under that hypothesis
- $P(B)$: marginal probability of the evidence
- $P(A \mid B)$: posterior probability after seeing the evidence

For text classification, the hypothesis is a class and the evidence is the document's features.

Prior, likelihood, and posterior

For a class c, combine its prior with the document likelihood:

$$\underbrace{P(c \mid \mathbf{x})}_{\text{posterior}} = \frac{\underbrace{P(\mathbf{x} \mid c)}_{\text{likelihood}} \underbrace{\pi_c}_{\text{prior}}}{\sum_{c'} P(\mathbf{x} \mid c')\, \pi_{c'}}.$$

The denominator normalizes the class scores so the posteriors sum to one.

The prior reflects class frequency before the text is read. The likelihood reflects how compatible the words are with that class.

If we deliberately balance a training corpus, its class frequencies may differ from those in the population where we apply the model.

Naive Bayes classification

Bayes' rule combines the class prior π_c with the document likelihood:

$$P(c \mid \mathbf{x}, n) \propto \pi_c P(\mathbf{x} \mid c, n).$$

The multinomial coefficient is the same for each class, so classification uses

$$\hat{c} = \arg\max_c \left[\log \pi_c + \sum_w x_w \log \theta_{cw} \right].$$

Logarithms turn a product of many small probabilities into a sum. Each observed word adds evidence toward one class or another.

What makes it naive?

The model treats tokens as independent draws from the class's word distribution, conditional on the class and document length.

It omits word order, syntax, and interactions such as *not good*.

The count vector itself has a fixed total. Its components are not independent, because they sum to the document length.

The simplifying assumptions make the model easy to estimate and inspect. Their adequacy must be checked on held-out texts.

Smoothing the word probabilities

Without smoothing, a term unseen in one class gets probability zero. One occurrence would then make that class's document likelihood zero.

$$\hat{\theta}_{cw} = \frac{n_{cw} + \alpha}{\sum_v n_{cv} + \alpha V}.$$

- n_{cw}: pooled training count of word w in class c
- V: number of terms in the fitted vocabulary
- $\alpha > 0$: additive smoothing, with $\alpha = 1$ giving add-one smoothing

Estimate counts from training texts. Choose the smoothing setting using validation data.

A worked three-word example

Training class	great	dull	plot	Total
Positive	8	2	5	15
Negative	1	7	7	15

With $V = 3$ and $\alpha = 1$, the denominator is $15 + 3 = 18$:

$$\boldsymbol{\theta}_+ = (9/18,\ 3/18,\ 6/18), \qquad \boldsymbol{\theta}_- = (2/18,\ 8/18,\ 8/18).$$

Each row sums to one. This is an invented example with equal class priors, $\pi_+ = \pi_- = 0.5$.

Predicting *great plot*

The count vector is $\mathbf{x} = (1, 0, 1)$. Omitting the common multinomial coefficient:

$$\text{Positive score} = 0.5(9/18)(6/18) = 1/12,$$
$$\text{Negative score} = 0.5(2/18)(8/18) = 2/81.$$

The positive score is larger. Normalizing gives

$$P(+ \mid \mathbf{x}) = \frac{1/12}{1/12 + 2/81} = \frac{27}{35} \approx 0.771.$$

This is a posterior under the small model, not a guarantee about a real review.

Naive Bayes is a learned weighted word rule

For two classes, take the difference between their log scores:

$$\log \frac{P(+ \mid \mathbf{x})}{P(- \mid \mathbf{x})} = \underbrace{\log \frac{\pi_+}{\pi_-}}_{\text{intercept}} + \sum_w x_w \underbrace{\log \frac{\theta_{+w}}{\theta_{-w}}}_{\text{learned word weight}} .$$

A word is positive evidence when its estimated class probability is larger in positive texts.

The structure resembles our dictionary score. The weights now come from labelled examples and smoothed class distributions.

Two approaches to classification

Naive Bayes

Estimate how each class produces words:

$$P(\mathbf{x} \mid c), \quad P(c).$$

Then use Bayes' rule to classify a new document.

Logistic regression

Estimate the probability of the label from the features:

$$P(c \mid \mathbf{x}).$$

Fit weights directly using labelled examples and a prediction loss.

Both can produce a linear classification boundary. They estimate their weights in different ways.

Logistic regression for text

Compute a weighted sum of the document features:

$$z_i = b + \mathbf{x}_i^{\mathsf{T}} \boldsymbol{\beta}.$$

Map that score to a probability:

$$p_i = P(y_i = 1 \mid \mathbf{x}_i) = \frac{1}{1 + \exp(-z_i)}.$$

$$z = -2 \Rightarrow p \approx 0.119, \quad z = 0 \Rightarrow p = 0.5, \quad z = 2 \Rightarrow p \approx 0.881.$$

The notebook uses TF-IDF features. The default binary rule predicts positive when $z_i > 0$, with ties assigned to the negative class.

What the fitting objective rewards

For one labelled example:

$$\ell(y_i, p_i) = -\left[y_i \log p_i + (1 - y_i) \log(1 - p_i)\right].$$

Predicted positive probability, if $y = 1$	Log loss
0.1	2.303
0.5	0.693
0.9	0.105

A confident incorrect prediction receives a large loss. Training finds coefficients that reduce the average loss across labelled examples.

Overfitting

A text model can have thousands of features and relatively few labelled documents.

It may learn accidental names, phrases, or source-specific patterns that happen to predict the training labels.

Look at training and validation performance together. A large gap suggests that fitting the training sample more closely is not improving generalization.

Regularization

Penalize large coefficients as well as prediction errors:

$$\mathcal{L}_{\text{reg}} = \frac{1}{N}\sum_{i=1}^{N} \ell(y_i, p_i) + \frac{\lambda}{2}\|\boldsymbol{\beta}\|_2^2.$$

- Larger λ encourages smaller weights.
- This reduces the influence of weakly supported features.
- Too much regularization can suppress useful differences.

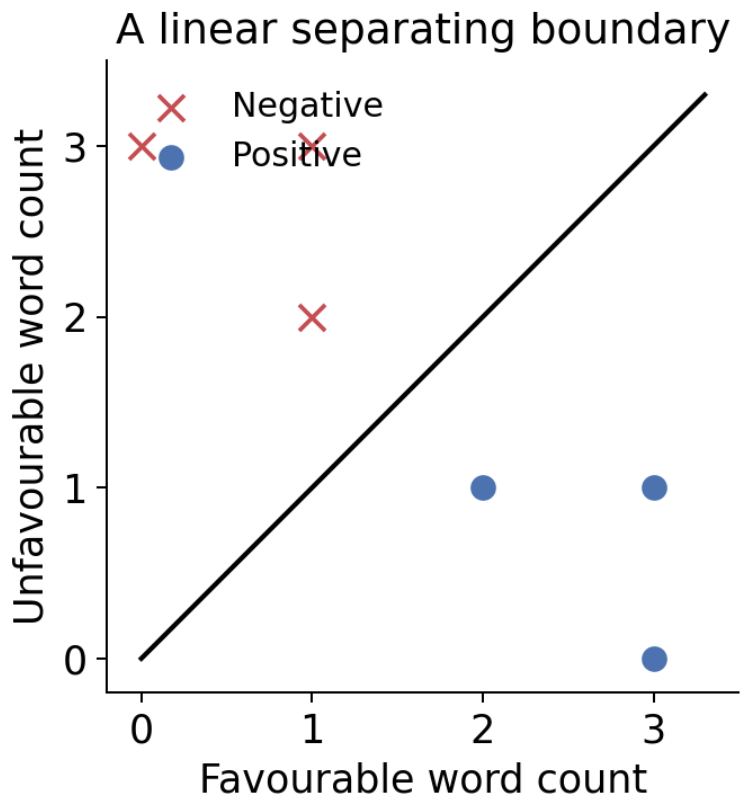
The notebook's software uses C, an inverse regularization setting: smaller C means a stronger penalty.

Hyperplanes in a feature space

A hyperplane is a flat boundary separating a feature space into two sides.

- In two dimensions, it is a line.
- In three dimensions, it is a plane.
- In a text vocabulary, it can involve thousands of features.

The plot uses two illustrative word-count features.



The hyperplane equation

For weight vector $\mathbf{w}$ and intercept b, the boundary is

$$\mathbf{w}^{\mathsf{T}}\mathbf{x} + b = 0.$$

The score gives the side of the boundary:

$$\hat{y} = \begin{cases} +1 & \text{if } \mathbf{w}^{\mathsf{T}}\mathbf{x} + b > 0, \\ -1 & \text{otherwise.} \end{cases}$$

Here the labels are written as -1 and $+1$ for the geometric explanation. The notebook reports them as 0 and 1.

Naive Bayes, logistic regression, and a linear SVM can all produce a boundary of this form. Their fitting rules differ.

Signed distance to the boundary

The score is proportional to distance. Divide by the length of the weight vector:

$$d_i = \frac{\mathbf{w}^\mathsf{T}\mathbf{x}_i + b}{\|\mathbf{w}\|}.$$

For $\mathbf{w} = (1, -1)$, $b = 0$, and $\mathbf{x} = (2, 1)$:

$$d = \frac{2 - 1}{\sqrt{1^2 + (-1)^2}} = \frac{1}{\sqrt{2}} \approx 0.707.$$

Multiplying $\mathbf{w}$ and b by the same positive constant leaves both the boundary and geometric distances unchanged.

A raw decision score is not a probability.

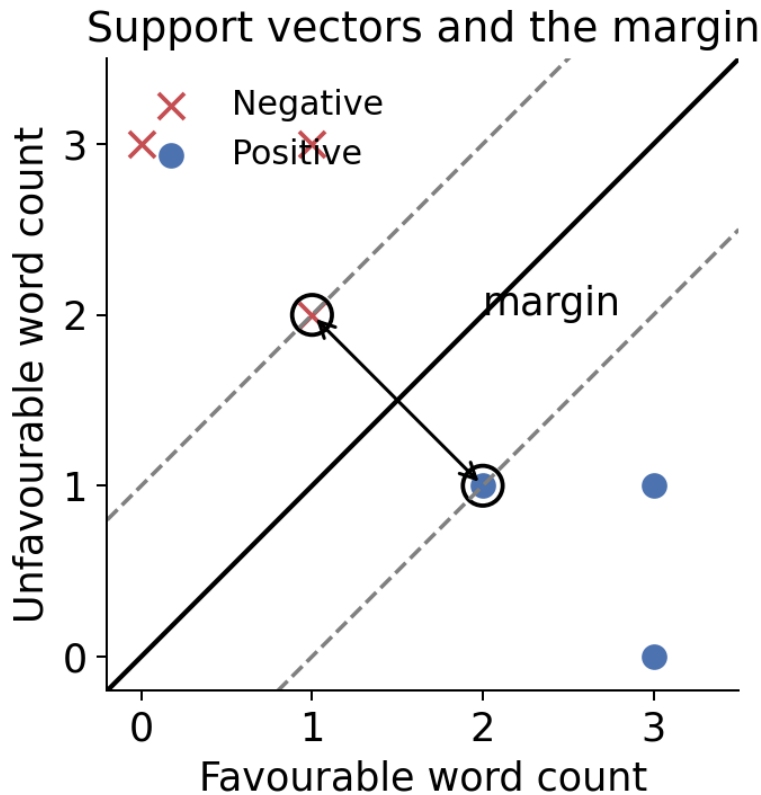
Support vectors and the maximum margin

For separable data, scale the weights so the closest points have scores $+1$ and -1.

$$\text{margin width} = \frac{2}{\|\mathbf{w}\|}.$$

The closest points are the support vectors.

The hard-margin SVM chooses the separator with the widest margin.



Soft margins for overlapping texts

Real texts rarely separate perfectly. Let $t_i \in \{-1, +1\}$ denote a training label and allow margin violations $\xi_i \geq 0$:

$$t_i(\mathbf{w}^\mathsf{T}\mathbf{x}_i + b) \geq 1 - \xi_i.$$

A standard soft-margin objective is

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \xi_i.$$

Larger C penalizes violations more strongly. Smaller C allows more violations to keep a wider margin.

With soft margins, influential points can be on the margin, inside it, or on the wrong side of the boundary.

Hinge loss

The margin condition can be expressed as a loss:

$$\ell_{\text{hinge}}(t_i, z_i) = \max(0,\, 1 - t_i z_i), \qquad z_i = \mathbf{w}^{\mathsf{T}}\mathbf{x}_i + b.$$

$t_i z_i$	Loss	Interpretation
1.5	0	Correct side, beyond the required margin
0.5	0.5	Correct side, inside the margin
-0.5	1.5	Wrong side of the boundary

This SVM loss differs from logistic log loss. Correct predictions can still have hinge loss when their margin is too small.

Linear SVMs for text

- Represent each text as a sparse vector, such as TF-IDF.
- Learn a linear boundary using a margin-based objective.
- Choose regularization strength using development data.
- Use the decision score to rank documents or apply a threshold.

Logistic regression transforms its score through a sigmoid. A linear SVM returns a decision score without that probability interpretation.

The notebook compares the two on the same unigram TF-IDF representation.

Parameters and settings

Estimated from training examples	Chosen through validation
Class-specific word probabilities	Naive Bayes smoothing α
Logistic coefficients and intercept	Regularization strength C
SVM weights and intercept	Margin penalty C
Vocabulary and IDF values	Minimum document frequency and n-gram range

Selecting settings is part of model development. The test set must not decide which setting we keep.

Interpreting a learned coefficient

A positive coefficient is evidence toward the positive label, conditional on the other features in this representation.

For a particular document, the contribution is

$$\text{contribution of feature } w = x_w \beta_w,$$

not just β_w by itself.

Coefficients are predictive associations. They do not estimate the causal effect of adding a word to a review.

Thresholds and probability scores

A model can rank texts well without producing well-calibrated probabilities.

Classification at a default threshold, ranking quality, and probability calibration are different properties.

Use validation data to choose a threshold appropriate to the research question. Use PR curves to inspect the precision–recall trade-off.

Average precision summarizes ranking. It does not verify that a probability of 0.8 is correct 80% of the time.

Generalization beyond this corpus

A classifier trained on movie reviews learns associations in movie-review language.

It may not interpret sentiment in parliamentary speeches, newspaper reporting, or social media the same way.

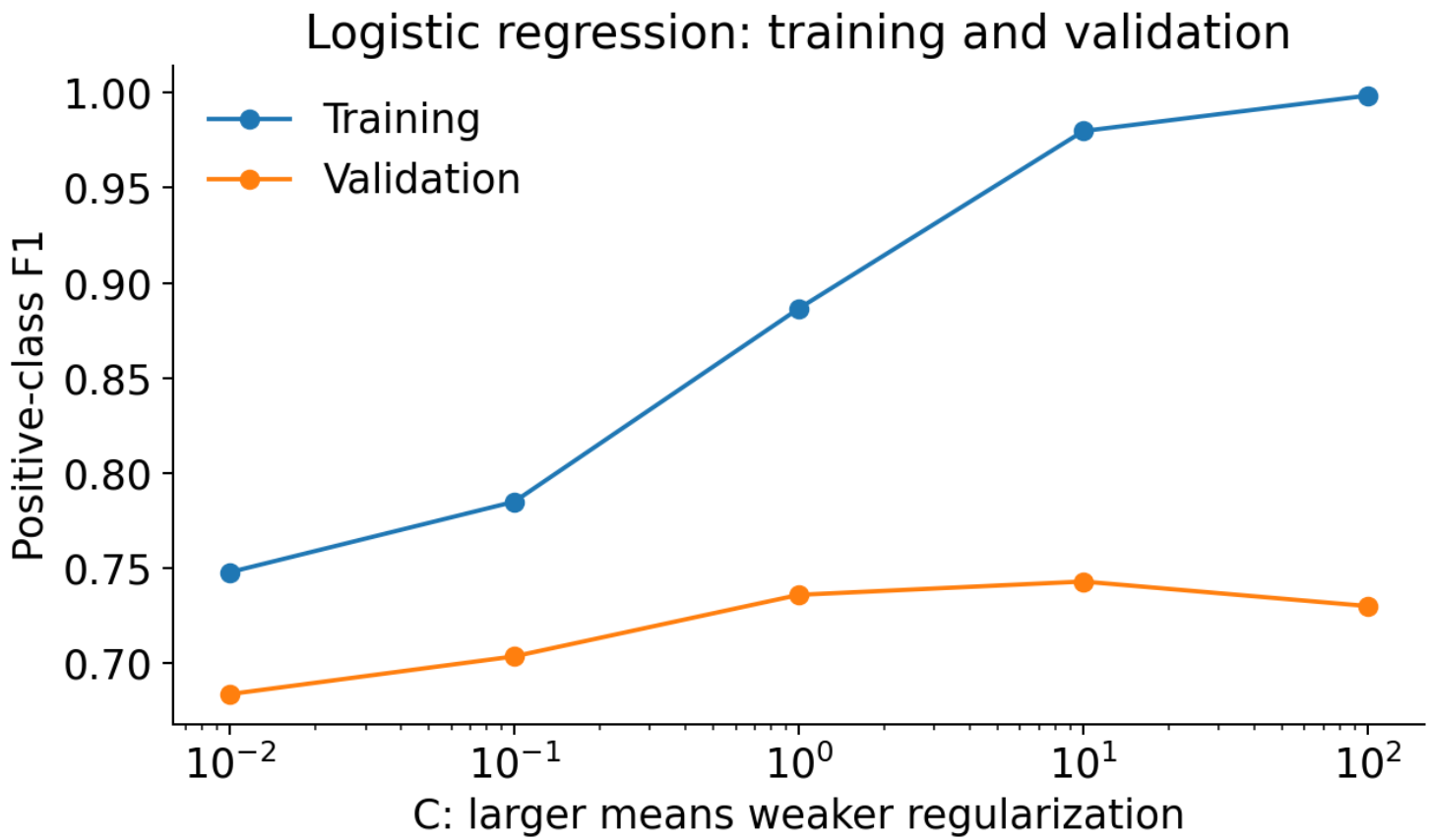
Changing the period, source, or genre can also change the meaning of a feature or label. Annotate and evaluate a sample from the target setting before treating the predictions as a measure.

Practical: a shared comparison

1. Keep the Class 4 dictionary as a fixed baseline.
2. Fit counts with multinomial Naive Bayes.
3. Fit TF-IDF with regularized logistic regression.
4. Fit a linear SVM on the same unigram TF-IDF representation.
5. Compare unigrams with unigrams plus bigrams.
6. Evaluate every rule on the same test snippets and inspect disagreements.

Vocabulary and IDF are fitted inside each pipeline. Stopwords are retained, and terms must occur in at least two training documents.

The validation curve in our notebook



The selected unigram setting is $C = 10$. We choose it by validation F1, then refit on the development data.

Results on the shared held-out test set

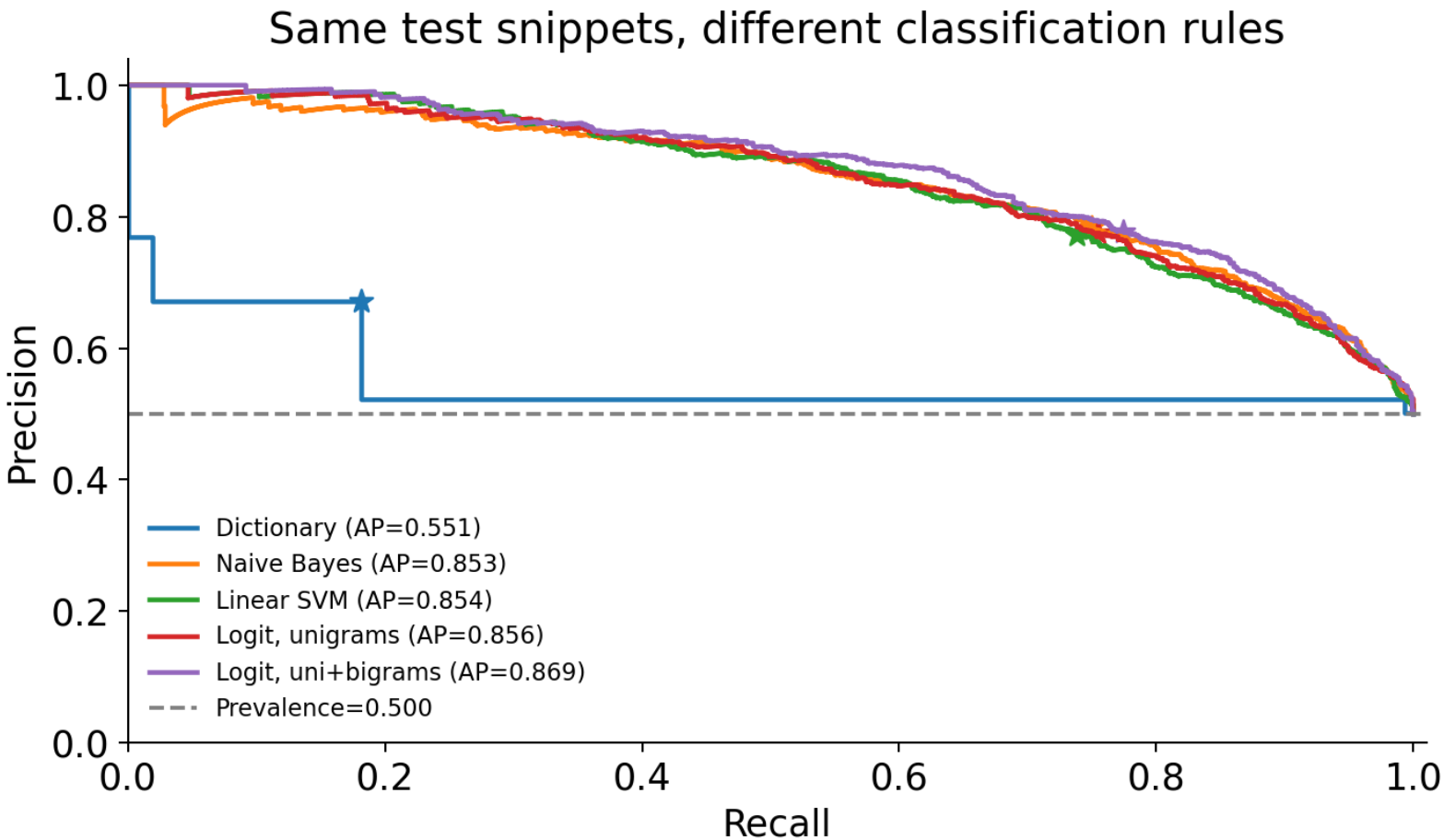
Method	Accuracy	Precision	Recall	F1
Majority	0.500	0.000	0.000	0.000
Dictionary	0.546	0.671	0.182	0.286
Naive Bayes	0.772	0.784	0.750	0.767
Linear SVM	0.760	0.773	0.739	0.755
Logit, unigrams	0.767	0.775	0.754	0.764
Logit, uni+bigrams	0.776	0.778	0.774	0.776

Precision, recall, and F1 use the positive class. The majority baseline's undefined positive precision is reported as zero.

Chosen settings: NB $\alpha = 1$; unigram logit $C = 10$; unigram–bigram logit $C = 10$; linear SVM $C = 1$.

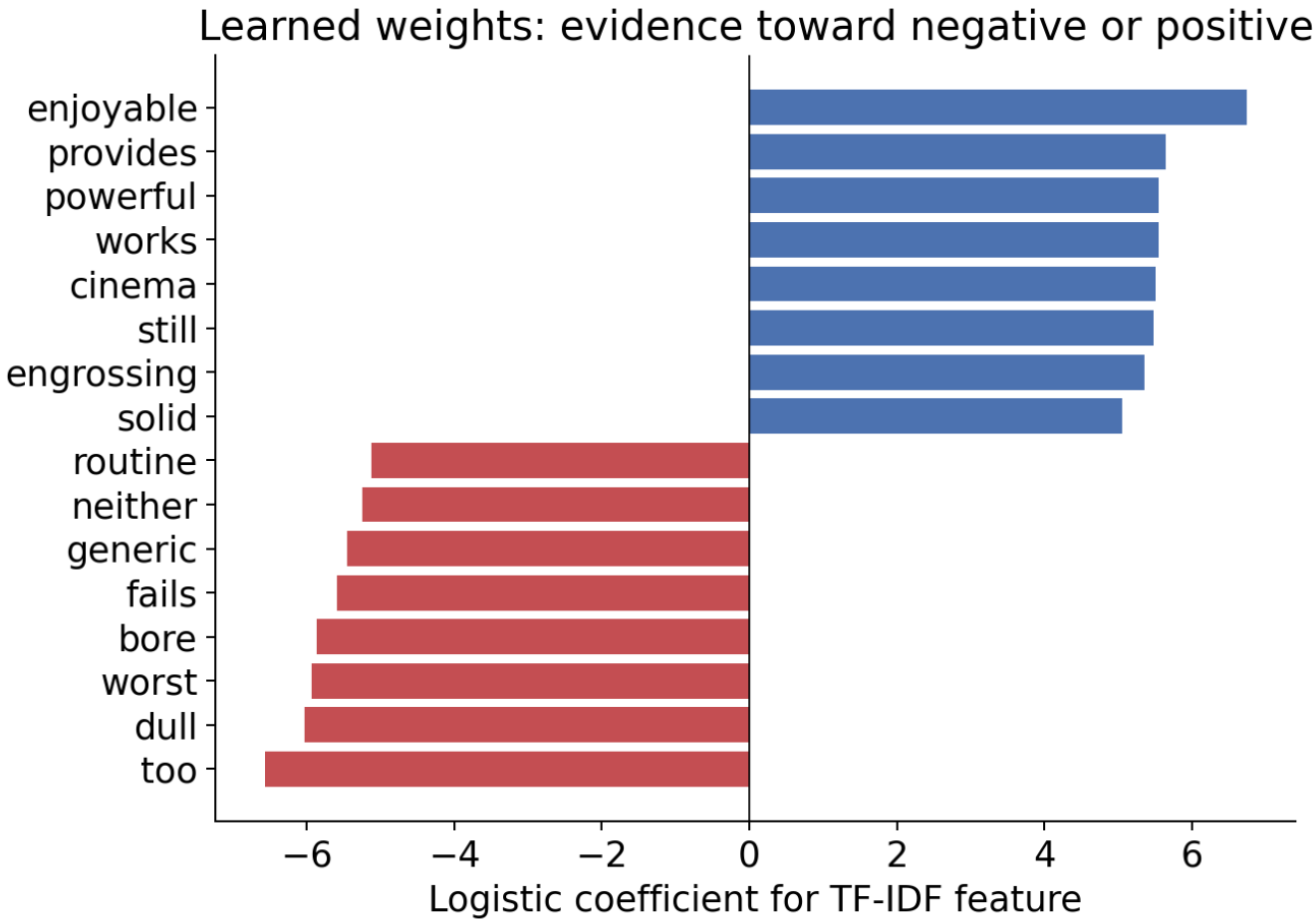
Computed in the Class 5 notebook. All methods use the same 2,133 test snippets.

Precision–recall comparison



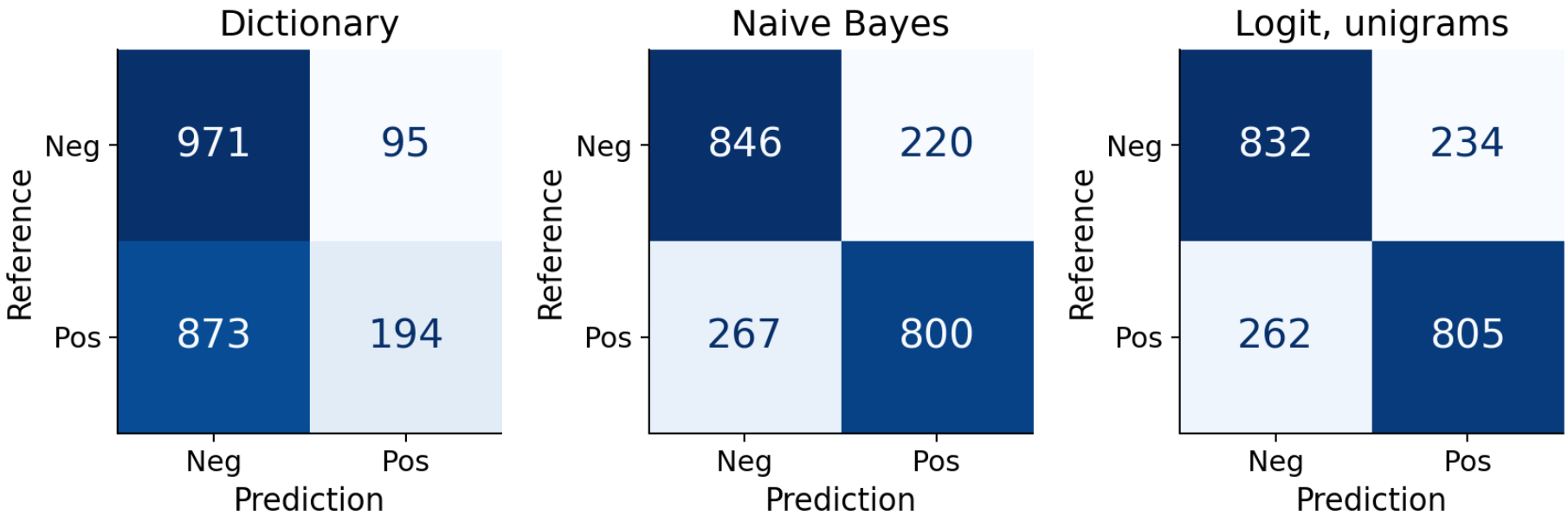
Stars show the default decision rules. These are final test curves, not a place to tune the threshold.

The words the logistic model learned



Weights describe predictive associations in the fitted unigram TF-IDF model. Compare these terms with the Class 4 dictionary.

Which errors changed?



Rows are source-review labels. Columns are model predictions. Read the false positives and false negatives as well as the total score.

Error inspection after model comparison

The notebook prints examples where the dictionary is wrong and logistic regression is right, followed by logistic regression's own errors.

For each disagreement, distinguish missing vocabulary, negation or irony, and a source-review label that is hard to infer from an isolated snippet.

A model improvement is informative when we can explain which errors improved and which remain.

Key takeaways

- Supervised classification learns a mapping from features to labels.
- Naive Bayes connects class-specific word probabilities to learned log-odds weights.
- Logistic regression learns weights through a prediction loss and regularization.
- Linear SVMs learn weights through a margin-based loss.
- Representation, data splits, and tuning are part of the method.
- Good prediction must still be connected to the substantive measurement.

Sources

- Pang, Bo, and Lillian Lee. 2005. *Seeing Stars: Exploiting Class Relationships for Sentiment Categorization with Respect to Rating Scales*. Proceedings of ACL. Sentence-polarity dataset v1.0.
- Grimmer, Justin, Margaret E. Roberts, and Brandon M. Stewart. 2022. *Text as Data*. Princeton University Press.
- Manning, Christopher D., Prabhakar Raghavan, and Hinrich Schütze. 2008. *Introduction to Information Retrieval*. Chapters 13–15.